Deep

1. Problem & MDP Formulation

Gardner Mini-Chess
5×5 board, 10 pieces per side
(K, Q, R, B, N, 5P)

State: 12 × 5 × 5 binary planes

- Ch 0-5: own {P,N,B,R,Q,K}
- Ch 6-11: opponent pieces
- Canonical view (mover's POV)

Q-Learning and Plann

A Comparison of DQN, D

Zhenkang Dong | CS 5180 Reinforcem

2. DQN & Double DQN

CNN Architecture:
Conv2d(12→64, 3×3) → BN → ReLU
Conv2d(64→128, 3×3) → BN → ReLU
Conv2d(128→128, 3×3) → BN → ReLU
FC(3200→512) → ReLU → FC(512→625)

ning in Self-Play Mini-

Double DQN, and MCTS

ent Learning | Northeastern University

3. MCTS (Planning)

Monte Carlo Tree Search with random rollout policy — no training required.

200 simulations per move decision.

-Chess

4. Self-Play Training

Training Pipeline:

① Warmup phase (ep 1-1000):
Agent vs Random opponent

② Self-play phase (ep 1001+):
Agent vs Frozen copy of itself

③ Frozen opponent refreshed

Actions: 625 (from_sq × to_sq)

- Legal-action masking

Rewards: Terminal only

- +1 win, -1 loss, 0 draw
- Move limit: 200 half-moves

5. Training Win Rate

```
DQN Target:
  y = r + γ · max_a' Q(s', a'; θ⁻)

Double DQN Target:
  a* = argmax_a' Q(s', a'; θ)   ← online
  y  = r + γ · Q(s', a*; θ⁻)     ← target
```

Key Hyperparameters:

- Learning rate: 1e-4 (Adam)
- Replay buffer: 100K transitions
- Target update: every 1000 steps
- ε-greedy: 1.0 → 0.05 (linear, 30K eps)
- Discount γ = 0.99
- Loss: Huber (Smooth L1)

6. Q-Value Analysis

```
function MCTS(state, N_sim):
  root = Node(state)
  for i = 1 ... N_sim:
    node = SELECT(root)   // UCB1
    node = EXPAND(node)
    v = ROLLOUT(node)     // random
    BACKPROP(node, v)
  return most_visited(root)
```

UCB1 Selection:
  score = Q/N + c · √(ln N_parent / N)
  c = √2 ≈ 1.414

Rollout: uniform random legal moves until terminal state.

Value stored from White's perspective at every node (no negation needed).

7. Evaluation Results

every 500 episodes

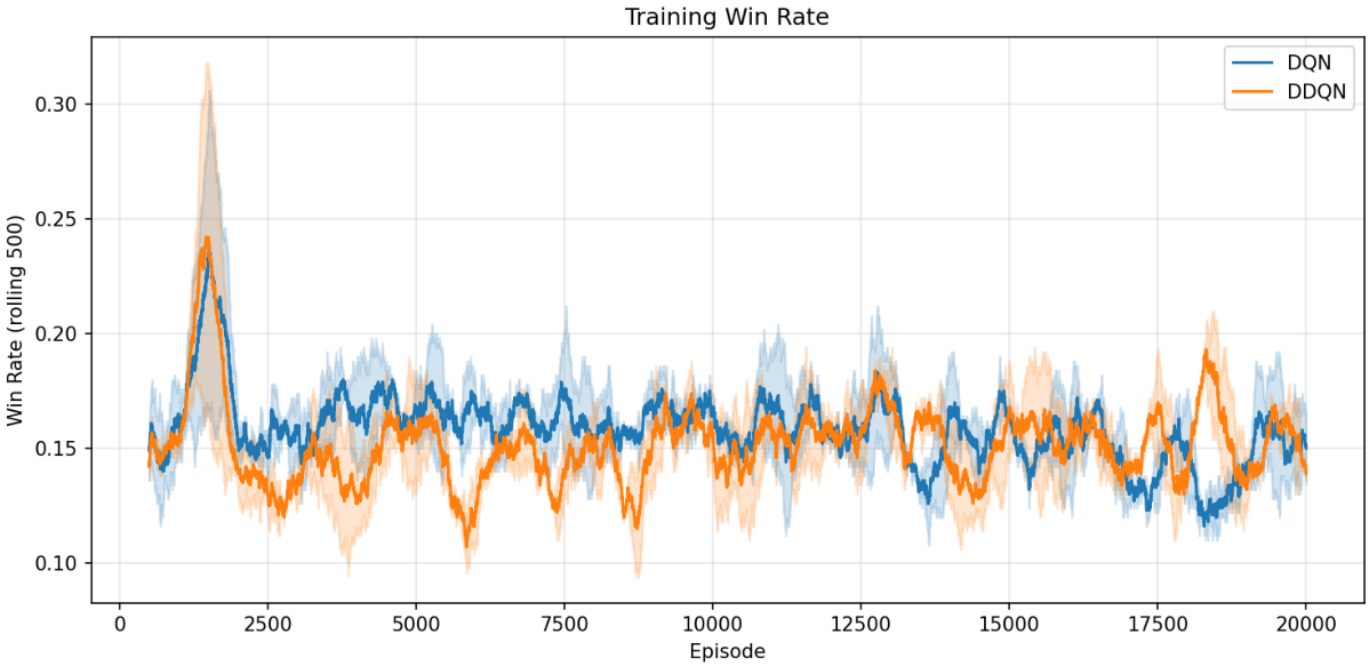
④ Colors alternate each episode
(mitigate first-move bias)

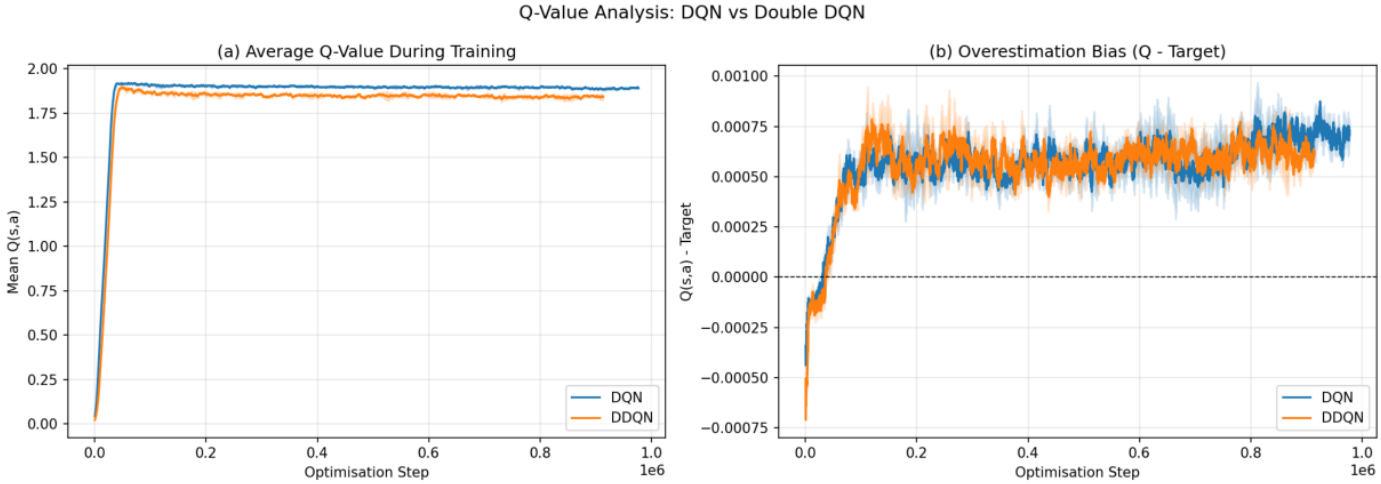
⑤ Canonical encoding:
Board always from mover's POV
→ same network plays both sides

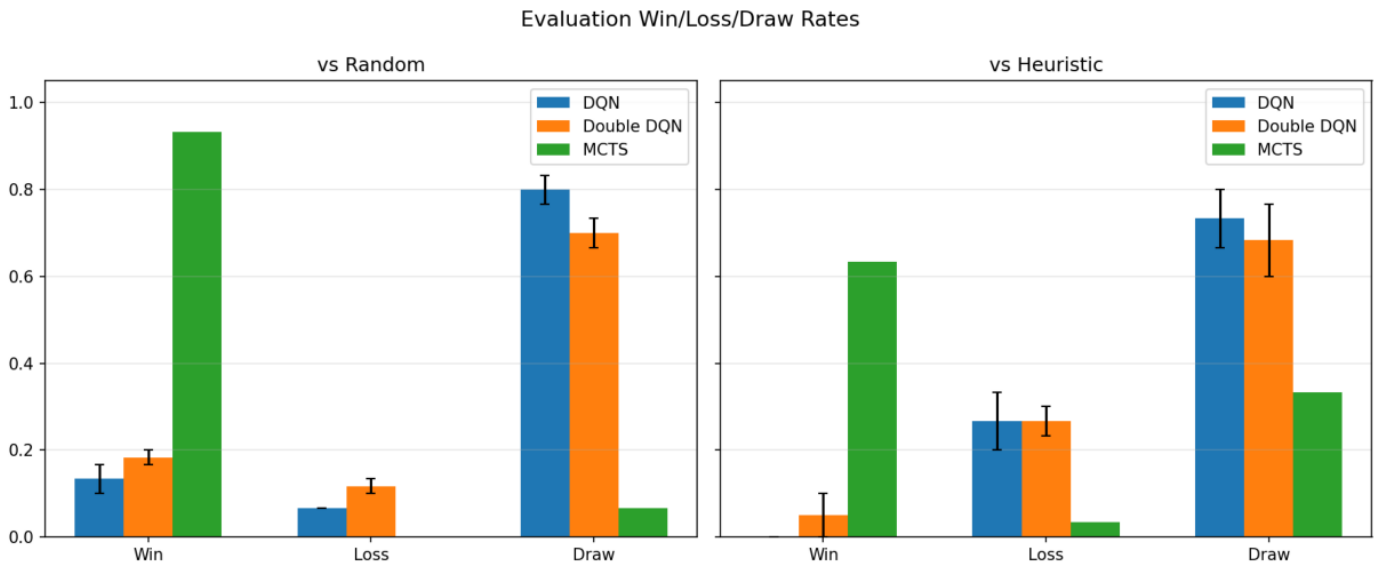
Training budget:
20,000 episodes × 2 seeds
(RTX 4060 Laptop GPU)

8. Conclusions

Key Findings:


Training Win Rate
DQN
DDQN
Win Rate (rolling 500)
0.30
0.25
0.20
0.15
0.10
0
2500
5000
7500
10000
12500
15000
17500
20000
Episode


Q-Value Analysis: DQN vs Double DQN
(a) Average Q-Value During Training
Mean Q(s,a)
Optimisation Step
1e6
DQN
DDQN
(b) Overestimation Bias (Q - Target)
Q(s,a) - Target
Optimisation Step
1e6
DQN
DDQN



	vs Random (W/L/D)	vs Heuristic (W/L/D)
DQN	13% / 7% / 80%	0% / 27% / 73%
DDQN	18% / 12% / 70%	5% / 27% / 68%
MCTS	93% / 0% / 7%	63% / 3% / 33%

- MCTS (93% win) greatly outperforms DQN (13%) and DDQN (18%) vs Random

- DDQN shows lower Q-value overestimation than DQN (avg Q: 1.84 vs 1.89)

- DDQN has lower variance across seeds (±0.8% vs ±1.4%)

- Both DQN/DDQN struggle with sparse terminal rewards (most games end in draw)

Future Work:

- Reward shaping (captures, checks)
- AlphaZero-style MCTS + NN
- Longer training (50K+ episodes)
- Policy gradient methods (PPO)